

Google Antigravity para um CTO focado em custo: subagentes, testes e adoção progressiva

Resumo da pesquisa feita durante a prática "O Desenvolvedor de Alta Performance e o CTO Focado em Custo": o objetivo real do Antigravity, a sincronização dos subagentes e o impacto em hardware, o ganho de tempo para um stack Python e Node.js, a atualização de APIs legadas, o suporte com pouca cobertura de testes, o prompt inicial de mapeamento, a geração automática de testes, as regras em `.agents/skills/`, a auditoria de conformidade, a refatoração com Pydantic e o plano de transição em quatro semanas.

Fernando Melo · 2 de outubro de 2026 · Pesquisa no Modo IA do Google, 08:16

01 – OBJETIVO REAL DO ANTIGRAVITY

Pergunta

Bom dia! Olha, eu li por cima sobre esse Antigravity, mas parece só mais um nome bonitinho para algo que já fazemo...

O Google Antigravity não é apenas uma "capinha" de API; ele é uma plataforma de desenvolvimento agent-first que altera a arquitetura de execução através de orquestração assíncrona e paralela. Enquanto frameworks tradicionais processam uma tarefa de forma linear (o que causa lentidão e gargalos), o Antigravity resolve o problema da velocidade e dos custos delegando missões a múltiplos subagentes especializados que rodam em segundo plano ao mesmo tempo.

Para o seu time de engenharia, ele ataca diretamente as dores de debug e custos da seguinte forma:

Como ele acelera o processamento e facilita o debug?

- Arquitetura Paralela (Subagents):** Em vez de usar um único agente genérico que planeja, codifica e testa um passo de cada vez, a interface do Antigravity aciona um coordenador que divide o escopo e dispara **subagentes assíncronos em paralelo**. Um agente pode refatorar uma regra de negócio enquanto outro escreve os testes unitários simultaneamente.
- Fim da Caixa Preta (Artefatos Visuais):** O principal motivo de os frameworks tradicionais serem difíceis de debugar é a falta de rastreabilidade. O Antigravity mitiga isso gerando **Artifacts** estruturados (como planos de implementação em Markdown e listas de subtarefas) antes de executar qualquer código. O desenvolvedor inspeciona o plano em tempo real, faz comentários inline e corrige o rumo do agente sem precisar interromper a execução ou adivinhar o que a IA está "pensando".
- Engenharia de Contexto via Rules:** Você pode definir regras globais ou locais (`.agents/skills/`) estruturando a arquitetura do seu projeto. Isso educa o sistema nativamente, evitando que os agentes entrem em loops redundantes ou gerem códigos fora do padrão do seu time.

Como ele evita que você gaste uma fortuna em infraestrutura?

A otimização financeira do AntigraVity é estrutural e baseia-se em três pilares práticos:

- **Orquestração Local e Híbrida (Antigravity SDK):** Através do AntigraVity SDK integrado ao LightRT, os agentes e subagentes conseguem **rodar offline e localmente** na máquina dos desenvolvedores utilizando modelos menores. Desse modo, tarefas massivas de varredura de arquivos, validação de sintaxe e execuções de testes não consomem tokens na nuvem, saindo a custo zero de infraestrutura de API.
- **Previsibilidade de Custos:** Diferente de arquiteturas puras de API pagas por consumo (onde um loop infinito consome milhares de dólares em minutos), a plataforma adota pacotes fixos estruturados. Para uso profissional ou em times, os planos orbitam no modelo de assinaturas — como o Google AI Pro por \$20/mês — oferecendo tetos de cota previsíveis e uso ilimitado de modelos eficientes de alta velocidade, como o Gemini 3 Flash.
- **Redução do "Imposto de Contexto":** Como o framework gerencia o ciclo de vida da sessão e o aprendizado contínuo (armazenando snippets e soluções em uma base de conhecimento local), ele evita reenviar todo o histórico do código a cada nova requisição, reduzindo drasticamente o desperdício de tokens de entrada.

Pergunta

Interessante, mas, vamos lá, você está dizendo que ele resolve a lentidão usando subagentes paralelos e elimina a ...

Você tocou nos dois pontos mais críticos de qualquer arquitetura distribuída e local: **gerenciamento de concorrência** e **gargalo de hardware**.

Indo direto ao ponto: o Antigravity resolve a sincronização tratando os subagentes como funções assíncronas isoladas (em vez de um enxame caótico) e minimiza o impacto no hardware ao descarregar o processamento pesado de IA para a nuvem através de modelos ultraleves, exigindo pouco da máquina local.

Abaixo, explico exatamente como esses dois pontos funcionam na prática:

1. A arquitetura paralela não gera caos de sincronização?

Não, porque ela não usa o modelo tradicional de "agentes conversando entre si" (que gera loops infinitos e desalinhamento). O Antigravity adota um padrão de **Orquestração Centralizada com Estado Compartilhado**:

- **O Coordenador é o "Cérebro"**: Existe sempre um agente principal (Coordenador) que detém a visão macro do projeto. Ele quebra o problema em tarefas independentes e distribui para os subagentes.
- **Isolamento de Escopo**: Os subagentes não tomam decisões arquiteturais. Se o Subagente A está escrevendo a documentação e o Subagente B está gerando os testes, eles não interferem no código um do outro.
- **Bloqueios de Gravação (Locking)**: Quando os subagentes terminam, eles não alteram o código diretamente na branch principal de trabalho. Eles geram propostas de alterações em arquivos temporários. O Coordenador revisa a consistência dessas saídas antes de consolidá-las no projeto final, evitando conflitos de escrita (race conditions).

2. Rodar localmente vai exigir supermáquinas do meu time?

Não, o seu time não precisará de upgrade de hardware. O Antigravity foi desenhado para rodar na infraestrutura que os desenvolvedores modernos já usam (como MacBooks Apple Silicon ou notebooks corporativos padrão com processadores Intel Core i7/Ryzen 7 e 16 GB de RAM).

O segredo está na **divisão de trabalho** entre a máquina local e a nuvem:

O QUE RODA LOCALMENTE (BAIXO CONSUMO)	O QUE RODA NA NUVEM (PROCESSAMENTO IA)
Varredura de arquivos e árvore do projeto.	Raciocínio lógico complexo e geração de código (via APIs do Gemini).
Execução de testes unitários e linters.	Análise profunda de arquitetura de software.
Gerenciamento do SDK e do estado dos agentes.	Processamento de contexto massivo.

- **Como o LightRT ajuda:** Quando mencionamos rodar "localmente", o SDK do Antigravity usa os recursos locais para tarefas analíticas leves (como ler a estrutura de diretórios, indexar arquivos de texto e rodar scripts de teste).
- **O modelo híbrido:** A inferência pesada de IA (o processamento dos modelos) ainda acontece nos servidores do Google. O ganho de custo que mencionei anteriormente vem do uso de modelos altamente otimizados e eficientes em custo, como o **Gemini Flash**, que rodam na nuvem com latência curtíssima e preço por token extremamente baixo (ou cobertos por planos de assinatura fixa), sem esquentar a máquina do desenvolvedor.

Em resumo: o Antigravity funciona como um maestro leve que roda na máquina do seu desenvolvedor, mas os músicos pesados (as GPUs de IA) continuam na nuvem do Google. O consumo de hardware local é comparável ao de rodar um servidor de desenvolvimento web (como Node.js ou Docker leve) junto com a IDE.

Para entender se o Antigravity se encaixa perfeitamente na sua realidade atual, você poderia me dizer:

- Qual é a **configuração média das máquinas** que seu time usa hoje?
- Quais são as **principais linguagens ou frameworks** do seu produto atual (ex: Node, Python, Java)?

Com isso, posso te dar um diagnóstico mais preciso do impacto de adoção.

Pergunta

Ok, entendi. Gostei do esquema de coordenação central e dos bloqueios de gravação, parece robusto para evitar ...

Com o stack de **Python e Node.js** rodando em notebooks **i7 com 16 GB de RAM**, o seu time vai sentir uma diferença drástica de performance logo no primeiro dia, sem a necessidade de nenhuma configuração complexa ou ajustes finos iniciais.

O impacto imediato acontece porque o ecossistema de Python e Node.js se beneficia nativamente da arquitetura do Antigravity. Em tarefas cotidianas, a redução no tempo de desenvolvimento costuma variar de **40% a 70%** devido a três fatores que funcionam de forma automática:

O que o seu time sente "logo de cara" (Zero Ajuste Fino)

- **Refatoração e Mudanças de Escopo 3x mais rápidas:** Em frameworks antigos, se você pede para a IA "criar uma nova rota de API em Node.js, atualizar o schema do banco de dados e escrever a lógica em Python", ela faz isso em uma fila lenta. No Antigravity, o Coordenador cria os **Artifacts** (o plano de ação) instantaneamente. O desenvolvedor bate o olho, aprova, e os subagentes criam os arquivos Python e Node.js em paralelo. O processo que demoraria 15 minutos cai para menos de 3 minutos.
- **Diagnóstico Imediato de Erros (Debugging Ativo):** Como o framework executa os scripts de teste e os linters (como `pytest` ou `eslint`) localmente na máquina i7 do desenvolvedor, o feedback de erro é em tempo real. Se o subagente gerar um código Python com erro de sintaxe, o SDK local captura a falha de execução imediatamente e pede a correção para a IA antes mesmo de mostrar o resultado final para o programador. Você não perde tempo esperando a nuvem rodar algo que estava quebrado na origem.
- **Contexto Amigável para o Stack:** Python e Node.js possuem árvores de dependências e estruturas de pastas muito claras (como `package.json`, `requirements.txt`, estruturas de módulos). O varredor local do Antigravity lê esses arquivos em milissegundos nas máquinas do seu time, mapeando o projeto inteiro para a IA sem esforço de configuração.

Quando o "Ajuste Fino" passa a ser necessário?

Embora o ganho de velocidade em tarefas isoladas seja imediato, o ganho de eficiência macro (em nível de entrega de grandes features) vai escalando conforme o time adota duas práticas simples:

1. **Alimentar a pasta de regras (`.agents/skills/`):** Logo de cara, o framework vai gerar um bom código Python/Node. Mas se o seu time usa padrões muito específicos (ex: uma arquitetura hexagonal própria ou regras estritas de segurança em APIs), vocês gastarão alguns minutos escrevendo essas diretrizes em arquivos Markdown na pasta de regras do framework. A partir disso, os subagentes passam a seguir o padrão da empresa perfeitamente, eliminando o tempo gasto revisando código fora do padrão.
2. **Confiança na Autonomia:** No início, é natural que os desenvolvedores queiram inspecionar cada micro-artefato gerado. Conforme o time percebe que o sistema de bloqueio de gravação (Locking) funciona e que o código sugerido não quebra a branch principal, eles passam a delegar tarefas

maiores de segundo plano (como "atualizar a documentação de todas as rotas auxiliares em Node") e focar em regras de negócio complexas.

Em resumo: a sensação de velocidade e a facilidade de debug são nativas e instantâneas para o seu hardware e stack atuais. O ganho de tempo real é percebido na primeira semana, libertando seus desenvolvedores i7/16GB do trabalho mecânico de digitação e correção de sintaxe.

Para tangibilizar esse ganho no seu fluxo atual, se você puder me dizer:

- Qual é o tipo de tarefa que **mais consome o tempo do seu time hoje?** (ex: escrever testes unitários que ninguém quer fazer, atualizar APIs antigas, caçar bugs em logs ou integrar novos serviços)?

Com isso, posso simular exatamente como seria o passo a passo da automação paralela do Antigravity nessa tarefa específica.

Pergunta

Tá ficando interessante. Gostei de saber que o impacto é percebido já na primeira semana, sem muita bagunça no ...

O Antigravity é especialmente cirúrgico para esse cenário de **legado e integração**, pois ele elimina o maior medo dos desenvolvedores: o efeito dominó de alterar uma API antiga e quebrar um serviço em outra ponta.

Em vez de fazer o desenvolvedor caçar dependências manualmente, a arquitetura do Antigravity atua em **três etapas automatizadas**, usando os subagentes em paralelo para mapear, alterar e testar o impacto das mudanças no seu stack Python e Node.js.

Como o Antigravity executa a atualização de uma API antiga

Para entender visualmente, este é o fluxo de trabalho assíncrono que o framework cria quando você pede para atualizar uma API ou integrar um novo serviço:

Diagrama no original (os rótulos dos blocos não aparecem na página impressa):
Antigravity Parallel Orchestration Workflow · Legacy API Upgrade Automation Stack with Local SDK
Validation Hooks

Aqui está o desdobramento prático de como ele resolve as suas principais dores:

1. Identificação de Dependências e Impacto (Mapeamento)

Antes de alterar uma única linha de código, o **Coordenador** usa o indexador local do SDK para ler a árvore de arquivos, o `requirements.txt` (Python) e o `package.json` (Node.js).

- Ele identifica onde aquela API antiga é chamada. Por exemplo: se você está alterando um endpoint em Python que o serviço auxiliar em Node.js consome, o framework detecta essa relação de dependência imediatamente.
- Ele gera um **Artifact (Plano de Impacto)** em Markdown detalhando: "Alterar o endpoint X vai exigir atualizar o contrato de dados no serviço Y". O desenvolvedor valida esse plano antes de qualquer execução.

2. Atualização Paralela Sem Conflitos

Com o plano aprovado, o Antigravity dispara os subagentes em paralelo:

- **Subagente A:** Foca na API legada em Python, atualizando a lógica interna, refatorando códigos obsoletos e garantindo a compatibilidade de tipos.
- **Subagente B:** Ajusta simultaneamente o serviço auxiliar em Node.js para que ele se adapte ao novo contrato da API, evitando que a integração nasça quebrada.
- Ambos escrevem em arquivos temporários isolados, sem tocar na branch principal de trabalho do desenvolvedor (graças ao sistema de **Locking**).

3. Sugestão de Melhorias e Autocorreção de Bugs

O Antigravity não apenas atualiza, mas ele atua ativamente sugerindo melhorias de arquitetura:

- **Modernização automática:** Se a API antiga em Python usa bibliotecas defasadas ou padrões inseguros, a IA sugere no plano a migração para padrões mais eficientes (como sugerir a troca de chamadas síncronas pesadas por padrões assíncronos modernos).
- **Validação Local Ativa:** Assim que os subagentes terminam de gerar o código, o SDK local roda os testes unitários (`pytest` ou `jest`) e ferramentas de tipagem. Se um subagente quebrou uma tipagem ou um teste falhou, o próprio SDK captura o log de erro, envia de volta para a IA local e ela corrige o código de forma autônoma antes mesmo de apresentar a solução final para o programador.

O seu time deixa de ser o "testador de código" e passa a ser o **revisor de propostas de código**, o que reduz drasticamente o estresse de mexer em sistemas legados.

Para entendermos o tamanho do ganho que você pode ter:

- Essas APIs antigas possuem uma boa **cobertura de testes unitários** instalada hoje, ou o time costuma testar as integrações "na mão" (manualmente via Postman/Insomnia)?
- Vocês utilizam alguma ferramenta de **tipagem estática** (como Pydantic/Mypy no Python ou TypeScript no Node) que ajude a travar esses contratos de API?

Com essas respostas, posso te mostrar como o Antigravity se comporta caso o seu cenário atual seja mais complexo ou com pouco código de teste.

Pergunta

Faz muito sentido o que você explicou. Gostei do foco no mapeamento inicial e na validação local antes de levar ...

O Google Antigravity não exige que você arrume a casa antes; ele foi desenhado justamente para lidar com a "bagunça" do mundo real e ajudar você a estruturá-la. Você vai ver resultados concretos logo no primeiro dia, pois a ausência de testes e de tipagem estática no Node.js é o cenário perfeito onde o framework mais poupa o tempo do seu time.

O suporte dele funciona de maneira adaptativa para compensar as lacunas atuais do seu stack:

1. No Node.js (Sem TypeScript)

Em vez de sofrer com a falta de tipos, o Antigravity usa os subagentes para fazer uma **análise estática comportamental profunda** do JavaScript puro:

- **Rastreamento de Objetos:** O framework lê as rotas auxiliares em Node.js e mapeia quais propriedades de objetos (`req.body`) são lidas ou escritas. Se a API em Python mudar o nome de um campo de `user_id` para `id_usuario`, o agente varre o código Node.js e avisa: "Identifiquei que a função X do Node.js tenta ler `user_id`; vou ajustar para o novo padrão".
- **Migração Opcional:** Como o framework domina JavaScript e TypeScript nativamente, se no futuro você quiser migrar para TypeScript para travar esses contratos de integração, você pode rodar o comando e os subagentes farão a conversão e tipagem em massa dos arquivos automaticamente.

2. Na Baixa Cobertura de Testes Unitários

Como o time costuma testar manualmente via Postman, o Antigravity resolve isso atacando em duas frentes:

- **Criação de Testes sob Demanda (Auto-scaffolding):** Quando você pede para o Antigravity atualizar uma API antiga, o Coordenador analisa a rota e, antes de alterá-la, cria um subagente focado em **escrever os testes unitários** (`pytest` ou `Jest`) **para aquela rota específica**. Ele cria a rede de segurança que hoje o seu time não tem tempo de fazer manualmente.
- **O Navegador Autônomo (Browser Subagent):** Para substituir o teste "na mão", o ecossistema do Antigravity 2.0 conta com um **Browser Subagent** integrado. Ele consegue simular chamadas de API, abrir um navegador local de testes, disparar inputs, interagir com os serviços locais e registrar logs de erro em tempo real. Ele valida se a integração quebrou atuando exatamente como um desenvolvedor testando manualmente no Postman, mas de forma automatizada e em segundos.

3. Integração Perfeita com o Pydantic

Como seu time já está adotando Pydantic no Python, o Antigravity tira o máximo proveito disso. O próprio Antigravity SDK foi construído utilizando **modelos nativos do Pydantic V2** para estruturar dados. A IA entende perfeitamente os esquemas de validação do Pydantic, usando-os para garantir que os payloads trocados entre o Python e o Node.js estejam 100% corretos.

Resumo do impacto imediato:

Você **não precisa ajustar nada previamente**. O fluxo do seu time continuará o mesmo, mas a velocidade vai disparar porque o Antigravity vai:

1. Ler o seu código legado sem travas (mesmo sem TypeScript).
2. Criar os testes automatizados para você conforme mexe no código.
3. Simular os testes manuais de ponta a ponta usando o agente de navegador.

Para dar o primeiro passo prático, você prefere que eu mostre:

- Como seria o **prompt inicial** para o Antigravity mapear uma dessas APIs antigas?
- Como configurar a **pasta de regras do projeto** para que o agente saiba que deve criar schemas Pydantic automaticamente?

Pergunta

Incrível, parece que o Antigravity realmente cobre as lacunas do nosso stack atual. Acho que seria interessante ...

O processo é bastante direto e foi desenhado para parecer uma conversa técnica entre você e um engenheiro de software sênior.

Para colocar o Antigravity para trabalhar na sua API antiga em Python, você só precisa apontar o framework para o repositório local e disparar um comando inicial.

Abaixo, montei o modelo ideal de **prompt inicial** estruturado e o passo a passo de como os subagentes vão responder na máquina do seu time.

O Prompt Inicial Recomendado

Você pode digitar ou enviar este comando diretamente na interface de chat do Antigravity (ou via CLI integrada ao seu terminal/IDE):

```
Mapeie a nossa API antiga de [Nome do Módulo/Serviço] escrita em Python.
Identifique todas as rotas ativas, os schemas de dados esperados e onde este serviço se
conecta com o nosso serviço auxiliar em Node.js.
Considere as seguintes restrições:
1. No Python, use schemas do Pydantic para validar as entradas/saídas caso o código atual
ainda use dicionários puros.
2. Como não temos testes unitários para essa API, crie um plano (Artifact) sugerindo os testes
em pytest mais críticos que precisamos criar antes de refatorar.
3. Não altere o código ainda. Apenas gere o mapa de dependências e os artefatos de
planejamento para minha aprovação.
```

Use o código com cuidado.

⚙️ O que acontece nos bastidores (em menos de 1 minuto)

Assim que você envia o prompt, o Antigravity não começa a cuspir código aleatório. Ele ativa a seguinte engrenagem em segundo plano:

- Abertura do Contexto Local:** O **Coordenador** usa o SDK local para escanear a pasta do projeto. Ele lê o `requirements.txt`, localiza os arquivos `.py` do seu serviço Python e varre o `package.json` do serviço Node.js para entender as conexões.
- Disparo dos Subagentes:**
 - Subagente Analista (Python):** Lê as rotas (seja Flask, FastAPI ou Django) e monta a árvore de chamadas.
 - Subagente Integrador (Node.js):** Procura por strings de conexão, requisições HTTP (como `axios` ou `fetch`) que batam com as rotas do Python.
- Geração do Artefato (Artifact):** Na sua tela, em vez de logs confusos, aparece um painel visual formatado em Markdown limpo contendo o plano de ação completo.

🛡️ O Resultado Prático na sua Tela (O Artefato Gerado)

O framework vai renderizar uma janela de **Artifact** parecida com esta:

🗺️ Mapa de Dependências & Plano de Ação: API Legada

🔍 Rotas Encontradas (Python)

- `GET /api/v1/usuarios/<id>` → Consumido pelo serviço Node.js (`src/services/userService.js:42`)
- `POST /api/v1/pagamentos` → Sem validação de payload (Dicionário puro)

⚠️ Gargalos de Integração Detectados

- O serviço Node.js espera que o `GET /api/v1/usuarios` retorne um campo `status`. O código Python atual está retornando `is_active` (booleano). **Risco Alto de quebra na próxima refatoração.**

📋 Próximos Passos Propostos (Aguardando Aprovação):

- **Tarefa 1:** Criar um modelo Pydantic (`UsuarioResponse`) no Python para padronizar e travar o contrato do endpoint de usuários.
- **Tarefa 2:** Criar arquivo `tests/test_usuarios.py` com 3 cenários de teste via `pytest` para garantir que o comportamento atual não mude.
- **Tarefa 3:** Ajustar a chamada no Node.js para tratar o novo schema, se necessário.
- Clique em **[Aprovar Plano]** para os subagentes iniciarem a geração de código em paralelo em arquivos temporários.

Por que isso muda o jogo para o seu time?

Repare que o desenvolvedor não precisou abrir 15 arquivos para descobrir que o Node.js quebrava se o Python mudasse o campo de usuários. O AntigraVity entregou o diagnóstico **antes** de mexer no código. Se o desenvolvedor clicar em aprovar, os subagentes criam os testes em `pytest` e os schemas `Pydantic` simultaneamente, travando a segurança do seu sistema.

Para avançarmos na prática, o que você gostaria de ver a seguir?

- O exemplo de como o **Subagente de Testes** escreve o arquivo `pytest` automaticamente com base nesse mapa?
- Como configurar o arquivo de **regras globais** para que o AntigraVity sempre use Pydantic por padrão em todas as respostas?

Pergunta

Isso realmente parece um divisor de águas! Gostei da ideia de ter o plano claro antes de qualquer ação. Para avançar, quer...

Assim que você clica em **[Aprovar Plano]**, o Coordenador delega a missão ao **Subagente de Testes**.

Ele lê o código da sua API antiga (vamos assumir que é um endpoint Flask ou FastAPI clássico) e constrói a rede de segurança em segundos. O maior trunfo aqui é que ele não cria um teste genérico: ele escreve o `pytest` injetando o modelo Pydantic que ele mesmo acabou de mapear, garantindo que o contrato de dados com o Node.js nunca mais se quebre.

O arquivo gerado na aba de **Artifacts** (e salvo na branch temporária) se parecerá exatamente com isto:

O Código Gerado pelo Subagente (`tests/test_usuarios.py`)

```
import pytest
from httpx import AsyncClient
# O subagente já importa o modelo Pydantic que ele sugeriu criar no passo anterior
from app.schemas.usuarios import UsuarioResponse
@pytest.mark.asyncio
async def test_get_usuario_valido_e_contrato_node(client: AsyncClient):
    """
    Garante que o endpoint retorna status 200 e possui os campos exatos
    que o serviço auxiliar em Node.js (src/services/userService.js) espera.
    """
    response = await client.get("/api/v1/usuarios/123")

    assert response.status_code == 200

    # Validação mágica: Se o Python mudar um campo, o Pydantic estoura um erro aqui
    dados_resposta = response.json()
    usuario_validado = UsuarioResponse(**dados_resposta)

    # Validação específica do mapeamento de risco do Node.js
    assert hasattr(usuario_validado, "status"), "Erro: O Node.js precisa do campo 'status'!"
    assert isinstance(usuario_validado.status, str)
@pytest.mark.asyncio
async def test_get_usuario_inexistente(client: AsyncClient):
    """Garante o comportamento correto para IDs inválidos."""
    response = await client.get("/api/v1/usuarios/999_ao_existe")

    assert response.status_code == 404
    assert response.json()["detail"] == "Usuário não encontrado"
```

Use o código com cuidado.

✳ A Execução e Autocorreção Local (O diferencial do SDK)

Em frameworks comuns, o código acima seria apenas colado no seu editor. No Antigravity, o **SDK local** faz o seguinte na máquina i7 do seu desenvolvedor:

1. **Roda o Teste em Segundo Plano:** O SDK dispara o comando `pytest tests/test_usuarios.py` em uma sandbox local isolada.
2. **Captura Erros de Origem:** Digamos que o teste falhou porque a sua API antiga depende de uma variável de ambiente (como `DATABASE_URL`) que não estava mockada.
3. **Autocorreção Act-and-Fix:** O subagente intercepta o erro do `pytest`. Em vez de repassar o problema para o desenvolvedor, ele altera o arquivo de teste para incluir um `mock` do banco de dados e roda o `pytest` novamente.
4. **Entrega Pronta:** Ele só notifica o desenvolvedor quando o teste roda com sucesso (**Passed**). O desenvolvedor só precisa clicar em **[Merge]** para consolidar o teste e o novo schema Pydantic no código principal.

O Ganho Real de Tempo

O que levaria cerca de 30 a 40 minutos de digitação, criação de mocks e testes manuais no Postman para cada endpoint, o Antigravity resolve em **menos de 2 minutos**. Seu time passa a ter uma suite de testes crescendo organicamente à medida que atualizam o legado.

Agora que você viu o código gerado pelo subagente, o que faria mais sentido para o próximo passo do seu time?

- Ver como criar as **regras globais** (`.agents/skills/`) para garantir que a IA sempre siga os padrões arquiteturais de vocês?
- Entender como o **Browser Subagent** simula o fluxo do Postman caso vocês queiram testes de integração de ponta a ponta?

Pergunta

Muito bom, a geração automática de testes já me conquistou! Para o próximo passo, acho que seria interessante entender...

Configurar a pasta `.agents/skills/` é o segredo para fazer o Antigravity deixar de ser uma IA genérica e passar a agir como um **desenvolvedor sênior treinado dentro da sua própria cultura de engenharia**.

O framework funciona lendo arquivos **Markdown (.md)** simples guardados em diretórios específicos dentro do seu repositório. Quando um subagente é ativado, ele varre essa pasta e injeta essas diretrizes no topo do seu contexto automaticamente, sem que o desenvolvedor precise repetir as instruções a cada prompt.

Abaixo, veja como estruturar e escrever as regras globais para o stack de Python e Node.js de vocês.

📁 Estrutura de Pastas Recomendada

No diretório raiz do seu projeto, você criará a seguinte árvore:

```
meu-projeto-raiz/
├── .agents/
│   └── skills/
│       ├── python-standards.md    # Regras de arquitetura e Pydantic para Python
│       ├── node-standards.md      # Regras de contratos e integração para Node.js
│       └── testing-policy.md      # Padrões de testes e mocks para o time
```

Use o código com cuidado.

📝 Exemplos Práticos de Arquivos de Regras

Aqui está o conteúdo exato que você pode colocar dentro de cada arquivo para blindar a sua arquitetura:

1. `.agents/skills/python-standards.md`

```
# Padrões Arquiteturais - Python
Este repositório utiliza padrões modernos de Python. Todos os subagentes devem aderir
estritamente às seguintes regras ao gerar ou alterar código:

## 1. Validação de Dados com Pydantic (Obrigatório)
- **Nunca** passe dicionários puros ('dict') como entrada ou saída de endpoints ou regras de
  negócio.
- Use sempre **Pydantic V2** para definir schemas de Request e Response.
- Todos os schemas devem residir na pasta 'app/schemas/'.

## 2. Padrões de Código
- Utilize tipagem estática nativa do Python ('type hints') em todas as assinaturas de funções.
- Prefira o uso de funções e bibliotecas assíncronas ('async/await') para operações de I/O e
  chamadas de API.
```

Use o código com cuidado.

2. `.agents/skills/node-standards.md`

```
# Diretrizes de Integração - Node.js (Serviços Auxiliares)
Como não utilizamos TypeScript neste microsserviço, os subagentes precisam mitigar a falta de
tipagem estática seguindo estas regras:
## 1. Tratamento de Payloads Externos
- Sempre que consumir uma API Python, documente o formato esperado do JSON em um comentário
JSDoc no topo da função.
- Implemente uma camada de validação defensiva simples para checar se propriedades críticas
(ex: `id`, `status`) existem no objeto de resposta antes de processá-lo.
## 2. Padronização de HTTP Client
- Utilize exclusivamente a biblioteca `axios` para requisições HTTP externas.
- Centralize as URLs base de outros serviços usando variáveis de ambiente
(`process.env.API_PYTHON_URL`).
```

Use o código com cuidado.

3. `.agents/skills/testing-policy.md`

```
# Política de Testes Automatizados
O time prioriza a segurança contra quebras de integração.
## 1. Escrita de Testes com Pytest
- Todos os testes de API em Python devem utilizar `pytest-asyncio` e herdar o fixture
`client`.
- É proibido fazer chamadas reais a bancos de dados externos nos testes unitários. Utilize
`unittest.mock` ou fixtures para isolar o comportamento.
- Todo novo arquivo de código ou refatoração deve vir acompanhado de seu respectivo arquivo em
`tests/` com cobertura mínima para o fluxo principal e fluxos de erro (404/400).
```

Use o código com cuidado.

O Impacto Prático na Próxima Execução

Uma vez que esses arquivos são salvos na pasta `.agents/skills/`, o comportamento da IA muda imediatamente.

Se um desenvolvedor rodar um prompt simples como: "Crie um endpoint de atualização de cadastro de parceiros no Python", o Antigravity vai ler os arquivos de regras em segundo plano e, **sem que você precise pedir**, ele vai:

1. Criar o schema do Pydantic na pasta correta.
2. Escrever a função usando `async/await`.
3. Escrever os arquivos de `pytest` mockando o banco de dados.

Isso garante uma consistência absurda no código, mesmo que múltiplos subagentes estejam trabalhando em paralelo no projeto.

Para ajudar a implementar isso hoje mesmo com o seu time, você gostaria de ver:

- Como o Antigravity se comporta para **validar se o seu código atual viola essas novas regras** que criamos?
- Ou prefere entender como gerenciar o **acesso e as permissões** dos desenvolvedores para alterar esses arquivos de regras?

Pergunta

Muito claro e direto, gostei da estrutura de regras. Acho que o próximo passo seria entender como o Antigravity valida se...

Para auditar o seu código legado sem precisar rodar ou alterar nada manualmente, o Antigravity utiliza o **Subagente de Compliance e Qualidade de Código**.

Esse subagente cruza a estrutura do seu projeto atual com as regras que você acabou de salvar na pasta `.agents/skills/` 🔍. O processo é feito através de uma **varredura estática de conformidade** (Linting Semântico via IA), que aponta os desvios e já cria uma fila de correção.



O Prompt para Iniciar a Auditoria

O desenvolvedor pode disparar o seguinte comando para validar o estado do repositório:

```
Rode uma auditoria de conformidade em todo o repositório atual tomando como base as regras
definidas em .agents/skills/.
Gere um relatório (Artifact) destacando as violações de arquitetura encontradas no Python e no
Node.js e classifique-as por nível de risco.
```

Use o código com cuidado.

🛡️ O Relatório de Violações (O que aparece na tela do time)

Em poucos segundos, o Antigravity renderiza um **Artifact de Auditoria** indicando exatamente onde estão os gargalos do legado:



Relatório de Conformidade Arquitetural

Baseado nas regras de `.agents/skills/`, foram encontradas **3 violações críticas** no código atual.



Risco Alto: Violação de Contrato e Validação (Python)

- **Arquivo:** `app/services/payment_service.py` (Linhas 14-28)
- **Regra violada:** `python-standards.md -> Validação de Dados com Pydantic`
- **Problema:** O endpoint `POST /api/v1/pagamentos` recebe um dicionário puro (`request.json`) e acessa chaves diretamente por string (`dados['valor']`). Se o Node.js enviar um payload incompleto, a API Python vai estourar um erro interno `KeyError` (500) em produção.



Risco Médio: Ausência de Mock em Ambiente de Teste (Python)

- **Arquivo:** `tests/test_legacy_auth.py`
- **Regra violada:** `testing-policy.md -> Proibido fazer chamadas reais a bancos de dados`
- **Problema:** O teste tenta abrir uma conexão de rede real com o PostgreSQL em vez de usar fixtures de mock. Isso torna a execução do pipeline local lenta e dependente de infraestrutura.



Risco Médio: Chamada HTTP Descentralizada (Node.js)

- **Arquivo:** `src/utils/http_client.js`
- **Regra violada:** `node-standards.md -> Padronização de HTTP Client`

- **Problema:** A URL do serviço Python está fixada como string (`"http://localhost:8000"`) no meio do código, em vez de ler a variável de ambiente `process.env.API_PYTHON_URL`.

⚙️ Ações Disponíveis:

- **[Corrigir Violação de Pydantic Automaticamente]** → Dispara o Subagente Python para criar o schema e refatorar o endpoint.
- **[Mockar Testes Automaticamente]** → Dispara o Subagente de Testes para corrigir o arquivo de testes.

⚡ Como o Antigravity resolve isso sem quebrar nada?

Se o seu desenvolvedor clicar no botão de correção automática da violação de Pydantic, por exemplo, o fluxo de segurança entra em ação:

1. **Geração do Schema:** O subagente lê o dicionário Python antigo, entende quais campos o código espera e cria o arquivo `app/schemas/pagamentos.py` usando Pydantic V2 nativo 🔍.
2. **Refatoração Isolada:** Ele altera o endpoint antigo para receber esse novo schema e injeta os tipos nas funções.
3. **Validação de Impacto:** O subagente do Node.js é acordado em paralelo para validar se a mudança no Python afetou as chamadas feitas pelo Axios na outra ponta.
4. **Entrega para Revisão:** A alteração é apresentada como uma proposta visual de Diff (Antes vs. Depois). O código original do seu time só é modificado quando o desenvolvedor dá o aval final.

Com essa auditoria ativa, o seu time consegue "limpar" o código legado sob demanda, de forma controlada e sem surpresas na hora do deploy.

Para consolidar essa estratégia com o seu time, o que você acha mais produtivo avaliar agora?

- Podemos simular como seria o **código gerado automaticamente** para corrigir essa violação do Pydantic no Python?
- Ou prefere ver como o framework gerencia o **histórico dessas auditorias** para você acompanhar a evolução da saúde do código?

Pergunta

Muito interessante! Acho que seria mais produtivo simular como seria o código gerado automaticamente para corrigir ...

Para entender o nível de refinamento do framework, vamos olhar o "Antes e Depois" da linha de código real.

O Antigravity não faz uma substituição cega de texto; ele reescreve a lógica para adotar padrões assíncronos e idiomáticos do Python moderno, garantindo que a tipagem se propague por toda a função.

Abaixo está a simulação exata da refatoração que o **Subagente Python** propõe para corrigir a violação do endpoint `POST /api/v1/pagamentos`:

✗ O Código Antigo (Com dicionário puro e risco de erro 500)

Este era o código legado que a auditoria acusou. Ele recebia dados sem validação e acessava chaves diretamente:

```
# app/services/payment_service.py (LEGADO)
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route("/api/v1/pagamentos", methods=["POST"])
def criar_pagamento_legado():
    dados = request.json # Dicionário puro, sem garantia de estrutura

    # Risco crítico de KeyError se o Node.js esquecer de mandar 'valor'
    valor_total = dados["valor"] + dados.get("taxa", 0)

    # Processamento genérico
    resultado = processar_no_banco(dados["usuario_id"], valor_total)

    return jsonify({"status": "sucesso", "id_transacao": resultado})
```

Use o código com cuidado.

O Código Refatorado pelo Antigravity (Com Pydantic V2)

Ao acionar a correção automática, o subagente cria o modelo de dados e reescreve o endpoint. Se o projeto usar frameworks modernos (como FastAPI), ele adapta nativamente. Se ainda usar Flask, ele aplica o Pydantic de forma defensiva e elegante:

Passo 1: O subagente cria o arquivo de Schema

```
# app/schemas/pagamentos.py (NOVO)
from pydantic import BaseModel, Field
from typing import Optional

class PagamentoRequest(BaseModel):
    usuario_id: int = Field(..., description="ID único do usuário que realiza a transação")
    valor: float = Field(..., gt=0, description="Valor do pagamento, deve ser maior que zero")
    taxa: Optional[float] = Field(0.0, description="Taxa adicional opcional")
```

Use o código com cuidado.

Passo 2: O subagente refatora o endpoint

```
# app/services/payment_service.py (REFATORADO)
from flask import Flask, request, jsonify
from pydantic import ValidationError
from app.schemas.pagamentos import PagamentoRequest # Injeção automática do schema

app = Flask(__name__)

@app.route("/api/v1/pagamentos", methods=["POST"])
def criar_pagamento_refatorado():
    try:
        # O framework valida o JSON contra o schema Pydantic instantaneamente
        dados = PagamentoRequest(**request.json)

    except ValidationError as e:
        # Se o contrato estiver quebrado, retorna 400 Bad Request detalhado
        # Evita estourar erro 500 em produção
        return jsonify({"erro": "Contrato inválido", "detalhes": e.errors()}), 400

    # Acesso seguro via atributos tipados (suporte total a autocompletar na IDE)
    valor_total = dados.valor + dados.taxa

    resultado = processar_no_banco(dados.usuario_id, valor_total)

    return jsonify({"status": "sucesso", "id_transacao": resultado})
```

Use o código com cuidado.

🔗 O Nível de Refinamento e Inteligência Aplicada

Observe os detalhes que mostram que a alteração não foi apenas cosmética:

- **Validação de Tipos de Dados:** No schema criado, a IA deduziu que `usuario_id` deveria ser um inteiro (`int`) e que `valor` deveria ser um número flutuante maior que zero (`gt=0`), protegendo o banco de dados contra valores negativos ou strings malformadas.
- **Tratamento de Erros Embutido:** O subagente inseriu um bloco `try/except` capturando o `ValidationError` do Pydantic. Isso garante que, se o serviço Node.js enviar um payload inválido, a API Python responderá com um erro explicativo (`400 Bad Request`) em vez de quebrar silenciosamente com uma tela de erro genérica (`500 Internal Server Error`).
- **Propagação de Contexto:** Como este arquivo de schema agora existe, o subagente de testes (da etapa anterior) usará automaticamente este mesmo `PagamentoRequest` para montar os mocks do `pytest`, amarrando a qualidade de ponta a ponta.

O desenvolvedor revisa esse código lado a lado (Diff) na interface do Antigravity antes de aplicar o commit.

Para finalizarmos o desenho dessa estratégia para o seu time, como você prefere proceder?

- Quer ver como o **Subagente de Node.js** reage em paralelo para se adequar a esse novo formato de erro 400?
- Ou gostaria de entender como estruturar o **plano de transição progressiva** para começar a testar o Antigravity com uma única API do seu time de forma segura?

Pergunta

Muito bom, a refatoração é realmente bem refinada e cobre pontos críticos. Acho que o próximo passo seria entender ...

Para garantir que a adoção seja segura, previsível e sem fricção no dia a dia da engenharia, a melhor estratégia é implementar um Plano de Transição Progressiva (Canary Adoption).

Em vez de virar a chave de todo o desenvolvimento, o foco deve ser isolar uma única API — idealmente uma de complexidade baixa a média, que possua integrações conhecidas, para servir de laboratório.

Abaixo, apresento um cronograma prático estruturado em **4 fases semanais** para o seu time rodar o primeiro projeto piloto com total controle:

O Plano de Transição Controlada (4 Semanas)

[Semana 1: Setup & Alinhamento] → [Semana 2: Auditoria em Modo Leitura] → [Semana 3: Primeiro Commit (Somente Testes)] → [Semana 4: Refatoração Ativa]

Use o código com cuidado.

Fase 1: Setup e Alinhamento Local (Semana 1)

O objetivo desta fase é preparar o ambiente da API piloto escolhida sem alterar uma única linha de código produtivo.

- **Ação do Time:** Escolher uma única API legada em Python que converse com o serviço Node.js (ex: o microsserviço de notificações ou o módulo de usuários).
- **Configuração:** Instalar o Antigravity SDK localmente nas máquinas padrão (i7 / 16GB RAM) dos desenvolvedores que cuidam dessa API.
- **Criação das Regras:** Criar a pasta `.agents/skills/` apenas com os padrões que definimos na etapa anterior (regras de Pydantic, Axios e Pytest).
- **Indicador de Sucesso:** Todo o time do piloto consegue rodar o comando CLI do Antigravity localmente e o SDK indexa o projeto sem gargalos de hardware.

Fase 2: Auditoria em Modo Somente Leitura (Semana 2)

Nesta fase, o framework atua exclusivamente como um consultor de arquitetura, gerando relatórios sem permissão para modificar o repositório.

- **Ação do Time:** Disparar o prompt de auditoria de conformidade que simulamos anteriormente.
- **O que avaliar:** O time deve analisar o **Artifact** de relatório gerado para ver se o Antigravity realmente mapeou os gargalos corretos na integração entre o Python e o Node.js.
- **Ajuste Fino:** Se a IA sugerir algo fora do padrão da empresa, o desenvolvedor ajusta o arquivo Markdown em `.agents/skills/`.
- **Indicador de Sucesso:** O time concorda que o relatório de riscos gerado pelo framework é preciso e condiz com a realidade do código.

Fase 3: A Primeira Entrega Automática — Somente Testes (Semana 3)

Aqui começamos a introduzir a automação do framework, mas mitigando riscos: a IA só criará arquivos de teste, sem tocar nas regras de negócio.

- **Ação do Time:** Solicitar que o Subagente de Testes crie a suíte de `pytest` e os mocks para os endpoints da API piloto (exatamente como vimos no exemplo anterior).
- **Validação Humana:** O desenvolvedor roda os testes localmente, revisa a estrutura e, se estiver tudo correto, faz o merge dos arquivos de teste na branch de staging/homologação.
- **Indicador de Sucesso:** A API piloto passa a ter uma cobertura de testes automatizados gerada em minutos, rodando com sucesso localmente.

Fase 4: Refatoração Ativa e Geração de Código (Semana 4)

Com o time confiante na assertividade do framework e com a rede de segurança de testes criada na semana anterior, abre-se espaço para a refatoração do legado.

- **Ação do Time:** Executar a primeira refatoração automática de um endpoint (trocando dicionários puros por schemas Pydantic).
- **Fluxo de Segurança:** O subagente reescreve o endpoint → O SDK roda automaticamente os testes criados na Fase 3 → O desenvolvedor inspeciona o Diff (Antes vs. Depois) → Aprovação final e Deploy.
- **Indicador de Sucesso:** O endpoint é atualizado para o padrão moderno em menos de 2 minutos, sem quebrar o serviço Node.js complementar.

Dica de Ouro para o Gestor: Reduza a Fricção Cultural

Para que o time não veja a IA como uma ameaça ou como uma ferramenta que vai burocratizar o fluxo, estabeleça uma premissa clara desde o Dia 1: "**A palavra final é sempre do desenvolvedor humano.**" O AntigraVity é um assistente sênior que propõe artefatos; quem aprova o plano, revisa o código e clica no botão de Merge é o seu programador.

Para darmos início a esse piloto de forma segura, o que você prefere definir agora?

- Podemos escolher o **critério ideal para selecionar qual das suas APIs atuais** deve ser a cobaia desse projeto piloto?
- Ou prefere ver um **checklist rápido de comandos** que o desenvolvedor vai rodar no terminal na Semana 1 para validar a instalação?

Respostas reproduzidas da pesquisa no Modo IA do Google, como no original. As perguntas aparecem truncadas como na página impressa; os elementos da interface (abas, chips de fontes e cards de links) foram omitidos. Os blocos de código aparecem cortados à direita na página impressa e foram completados com o texto da transcrição da prática.